

LLMOrchestrator

LLMOrchestrator

あらゆるヘッドレスCLIコーディングエージェントのための単一コントロールプレーン

概要

LLMOrchestratorは、スタンドアロンかつ再利用可能なGoモジュールであり、ヘッドレスCLIEージェント（OpenCode、Claude Code、Gemini、Junie、Qwen Code）を生成・管理し、ハイブリッドなパイプ+ファイルプロトコルを介して通信を行うためのものです。エージェントごとのサーキットブレーカー、プラグブルなマルチプロバイダー選択機能、疎結合のi18n抽象化レイヤー、そしてブラフ防止テスト保証を備えています。

短い説明

再利用可能なGoモジュールで、ハイブリッドなパイプ+ファイルプロトコルを通じて複数のLLM駆動のCLIEージェントを生成・制御するための統一インターフェースを提供します。スレッドセーフなエージェントプーリング、サーキットブレーカー、選択可能なルーティング戦略を備え、意図的にコンシューマー非依存で設計されており、プラグブルなi18nトランスレーターを搭載しています。

長い説明

LLMOrchestratorは、ヘッドレスCLIコーディングエージェントをオーケストレーションするための共有インフラストラクチャーです。マルチエージェントシステムに必要不可欠でありながら、各プロジェクトが独自に実装しがちなプロセス生成、メッセージフレーミング、結果解析といった機能を、OpenCode、Claude Code、Gemini CLI、Junie、Qwen Codeなどのツールに対して統一的に提供します。具体的には、統一されたAgentインターフェース、スレッドセーフなAgentPool、そして複数のプロバイダーからエージェントを統合管理するMultiProviderPoolを提供します。ルーティングはAgentSelectorを通じてプラグブルに設計されており、要件を満

たさないプロバイダーをスキップするラウンドロビン方式や、フォールバック付きの優先順位方式など、作業の分配方法は選択可能なポリシーとして実装されています。各エージェントは共通のBaseAdapterを基盤とするシンプルなアダプターとして実装されており、プロセスライフサイクル全体（パイプのセットアップ、SIGTERM→SIGKILLによる優雅な停止、再起動、生存確認など）を管理します。これは煩雑でエラーが発生しやすい部分ですが、一度解決されています。

通信は意図的にハイブリッド方式を採用し、用途に応じた最適なトランスポートを選択します。パイプトランスポートは、リクエストごとに読み取りデッドラインとレスポンス長制限を設けた改行区切りのJSONを使用し、高速なインタラクティブメッセージングを実現します。一方、ファイルトランスポートは、セッションごとのインボックス／アウトボックス／共有ディレクトリを使用し、パイプに収まらない大容量または永続的なデータを扱います。耐障害性は後付けではなく、構造的に組み込まれています。具体的には、エージェントごとに3回連続で失敗すると60秒間のクールダウンを経てハーフオープン状態で再試行するサーキットブレーカーを備え、バックグラウンドのヘルスマonitorがエージェントの生存確認を行うため、ダウンしたエージェントは受信トラフィックを待たずに復旧できます。プールの取得はCPUを無駄に消費するビジーウェイトではなく、条件変数を用いてブロックします。また、レスポンスパーサーはステートレスで、並行呼び出しに対しても安全です。モジュールは厳密に疎結合設計が徹底されており、コンシューマー固有の要件が入り込むことはありません。また、ユーザー向けの文字列はすべてプラグブルなi18n Translatorを通じて処理され、NoopTranslatorを使用することで翻訳が存在しない場合にはメッセージIDがそのまま表示され、問題が隠蔽されることなく明示的に示されます。

開発の背景

マルチエージェントシステムでは、CLIEージェントを確実に起動し、通信を行う必要があります。プロジェクトごとにプロセス生成、フレーミング、解析、障害対応を再実装することは非効率的でエラーの原因となります。LLMOrchestratorはこれらを一つの疎結合で再利用可能なモジュールに集約し、その専門性によって再利用性を確保しています。そして、その再利用性は、コンシューマー固有の要件が入り込む瞬間に失われてしまいます。

コンテンツ

なぜこれはゲームチェンジャーなのか

「異種CLIEージェントの軍団を動かす」という作業が、プロジェクトごとの特注エンジニアリング作業から、単なるライブラリのインポートへと変わります。プーリング、サーキットブレーカー、ライフサイクル管理、プラグブルなルーティングといった問題はすでに解決済みで、実績も十分です。さらに、そのアンチブラフテストは「コンパイルが通る」だけで満足

せず、実際のシステムをエンドツーエンドで検証するため、並行処理や障害発生時にも信頼できる抽象化が手に入ります。単に図面上で正しそうに見えるだけのものではありません。

革新的なポイント

- ハイブリッドパイプ+ファイルプロトコル – インタラクティブな速度 (JSON行のstdin/stdout、読み取りデッドライン、レスポンス上限) と、耐久性のあるファイルベースの交換 (インボックス/アウトボックス/共有) を両立。レイテンシと耐久性をトレードオフする必要はありません。
- マルチプロバイダープールとプラグブルセクター – 複数のCLIプロバイダーを統合する単一のファサード。ラウンドロビンや優先順ルーティンギは、ハードコードではなくポリシーで選択可能です。
- エージェントごとのサーキットブレーカー+バックグラウンドヘルスマニター – 自動的な劣化と復旧 (3回の失敗で60秒間オープン → ハーフオープンプロブ)。不安定なエージェントは隔離され、手動介入なしで静かに復帰します。
- ノンビジーウェイトプーリング – Acquireは、sync.Cond上でブロックし、条件に合った健全なエージェントが解放されるか、コンテキストがキャンセルされるまでCPUを消費しません。
- 厳密な疎結合+アンチブラフ **i18n** – NoopTranslatorはメッセージIDをそのまま返すため、翻訳漏れがあっても空白でごまかされることはありません。
- セキュリティ・バイ・デフォルト – バイナリパスの許可リストにより、シェル補完やコマンドインジェクションのリスクを排除。さらに、パストラバーサル対策、1MiBのレスポンス上限 (暴走出力防止)、APIキーのログマスキングを実装。
- アンチブラフ・チャレンジハーネス – 5つのロケール (en/sr/ja/es/de) で実際のディスク/JSON/パーサーのラウンドトリップを実行。機能が壊れた際には必ず非ゼロで終了するミュートーションゲートを併用し、テスト自体が機能することを証明します。

最大の技術的課題とその解決策

- 信頼性の高いエージェントプロセス **I/O** – スポーンされたCLIプロセスとの通信は一見簡単そうで難しい。ハイブリッドパイプ+ファイル転送、メッセージ/パーサーコントラクトによるワイヤーフォーマットの合意、BaseAdapterによるプロセスライフサイクルの一元管理 (SIGTERMタイムアウトからSIGKILLフォールバックまで含む) で解決。
- ビジーウェイトなしの並行処理 – ミューテックス+条件変数を用いたAgentPoolで、Acquireは条件に合ったエージェントが実際に解放されるまでスリープ。ステートレスで副作用のないパーサーにより、複数のルーチンからの安全な呼び出しを実現。
- プロバイダー障害の隔離 – 1つの不良プロバイダーが他を巻き込まないように、エージェントごとのサーキットブレーカーで影響範囲を限定。へ

ルスモニターゴルーチンにより、リクエストがなくても自動的に復旧を促進。

- コンパイルだけでなく正しさの証明 – チャレンジャーランナーで、en/sr/ja/es/deの数十の不変条件を実際のシステムで検証。さらに、ミュレーションゲート（LLMORCH MUTATE RUNNER=1が失敗する → ラッパー終了コード99）により、機能が壊れた際にゲート自体がブラフでないことを証明。
- 翻訳漏れの不可視化防止 – メッセージIDをそのまま返す
NoopTranslatorの仕組みと、コンシューマーごとの翻訳インジェクションにより、翻訳の欠落が常に可視化されるように解決。

テックスタック

- **Go**（バージョン1.25） – 優れた並行処理とクリーンなプロセス制御を備え、ライブエージェントプロセスのオーケストレーションにまさに適した選択。モジュール本体、エージェントアダプター、トランスポート、パーサーを実装。
- **Go標準ライブラリのみ（+ testify、yaml.v3）** – 依存関係を最小限に抑え、LLM SDKを一切導入しないという意図的な選択。これにより、モジュールは軽量かつ埋め込み可能となり、利用者側にベンダー固有の負担を強いることがない。
- **パイプトランスポート（stdio上のJSON-lines）** – 高速なインタラクティブメッセージングに最適。読み取りデッドラインとレスポンス長制限を設けることで、ハングアップや暴走したエージェントが呼び出し元を停止させるリスクを排除。
- **ファイルトランスポート（inbox/outbox/共有）** – セッションごとの耐久性と大容量アーティファクトの交換に適した選択。パイプでは不向きな用途に対応。
- **sync.Mutex/sync.Cond** – ビジーウェイトなしでブロッキングかつ公平なエージェントプールの取得を実現。
- **サーキットブレーカー + ヘルスモニター** – エージェントごとの耐障害性と能動的な復旧を両立。単なる障害検知にとどまらない。
- **pkg/i18n Translator** – コア部分から利用者固有の文字列を切り離す、疎結合なローカライゼーションレイヤーとして採用。
- **チャレンジハーネス（challenges/runner） + Makefile（test-race、fuzz、cover）** – レース検出やパーサーファジングを含む、対抗条件下での正確性を実証するための厳格な検証フレームワーク。根拠に基づく検証を重視し、推測に頼らない。

ステータスと正直な注意点

- ステータス：ベータ版。複数のHelix/vasicプロジェクトでサブモジュールとして利用される、疎結合な再利用可能モジュール。ライセンス：**Apache-2.0**。GitHubリポジトリは公開済み。
- モデルメタデータはHelixQA経由でLLMsVerifierから提供される。本モジュールはLLMsVerifier/VisionEngine/DocProcessorを直接インポートしない。親アプリケーションのCLAUDE.md（Gin/PostgreSQLな

ど)に記載されるスタックはhelix_codeを指し、本モジュールとは異なる。

優先度: Helixプライマリ (LLMインフラストラクチャクラスター — 疎結合な再利用可能モジュール)。HelixTrackに次ぐ優先順位。